

Top 50 Backend Interview Questions

NODE.JS, EXPRESS.JS, & CORE BACKEND • JUNIOR DEVELOPER LEVEL

Section 1: Core Node.js & Architecture

1. What is Node.js, and how does it differ from a browser-based JavaScript environment?

Node.js is a runtime that executes JavaScript outside a browser. Browsers interact with the DOM, whereas Node.js provides APIs to interact directly with the operating system and file system.

2. What does it mean that Node.js is "single-threaded"? How does it handle multiple tasks?

It executes all JavaScript code on a single primary thread. It handles multiple asynchronous operations concurrently by offloading them to the system kernel or Libuv's thread pool.

3. Explain the Event Loop in Node.js and its main purpose.

The Event Loop is a continuous mechanism that monitors and orchestrates asynchronous tasks. Its purpose is to offload blocking tasks to the background and run their callbacks on the main thread when finished.

4. What is the difference between asynchronous (non-blocking) and synchronous (blocking) code?

Synchronous code runs sequentially, freezing execution until the current line finishes. Asynchronous code registers a background task and immediately runs the next line, handling results later via callbacks.

5. What is the Libuv library, and why is it important in Node.js?

Libuv is a multi-platform C library that provides the core Event Loop engine for Node.js. It maintains a background thread pool used to execute heavy, blocking I/O operations like file manipulation.

6. What is the difference between `process.nextTick()` and `setImmediate()`?

`process.nextTick()` executes its callback immediately after the current operation finishes, before the Event Loop moves to the next phase. `setImmediate()` runs during the "Check" phase of the next full Event Loop cycle.

7. What are Streams in Node.js, and why are they efficient?

Streams are structures that process data piece-by-piece (chunks) over time instead of loading entire files into RAM. They are efficient because they allow manipulation of massive files with minimal memory usage.

8. Explain the difference between `require()` (CommonJS) and `import` (ES Modules).

`require()` is the legacy syntax that loads modules synchronously at runtime. `import` is the modern ES6 standard that parses modules statically at compile-time, allowing optimization like tree-shaking.

9. What is the purpose of the `package.json` file in a Node project?

It acts as the manifest file for your project, storing essential metadata and script commands. It tracks the explicit third-party dependency packages required to build and run the application.

10. What is the difference between dependencies and devDependencies?

`dependencies` are packages required to run the core application in production (e.g., Express).

`devDependencies` are only needed during local development and testing phases (e.g., Nodemon, Jest).

Section 2: Express.js Fundamentals

11. What is Express.js, and what primary advantages does it offer over native Node.js?

Express.js is a minimal and flexible web application framework built on top of Node.js. It simplifies backend development by providing abstractions for routing, HTTP requests, and middleware handling.

12. What is Middleware in Express? Give a practical example of why you would use it.

Middleware are functions that have access to request and response objects to intercept, modify, or block traffic. A practical example is an authentication check that verifies a user is logged in before routing.

13. What is the purpose of the `next()` function in Express middleware?

The `next()` function signals to Express that the current middleware has completed its processing. It hands control over to the next matching route handler, preventing the request from hanging indefinitely.

14. Explain the difference between app-level middleware and router-level middleware.

App-level middleware applies globally to every single route across the entire application via `app.use()`. Router-level middleware is isolated strictly to specific router instances to handle sub-routes.

15. How do you extract values from Route Parameters vs. Query Strings in Express?

Route parameters are dynamic variables embedded in the URL path, accessed via `req.params`. Query strings are optional key-value pairs appended after a question mark in the URL, accessed via `req.query`.

16. What does `app.use(express.json())` do, and why is it placed at the top of a file?

It is a built-in middleware that parses incoming raw JSON payloads and populates them into the `req.body` object. It sits at the top so that data is fully parsed before hitting route handlers.

17. How do you handle centralized error routing natively in Express?

You create a specialized middleware function that accepts exactly four arguments: `(err, req, res, next)`. Express automatically routes any error passed into a `next(error)` call directly into this handler.

18. What is the difference between `res.send()`, `res.json()`, and `res.end()`?

`res.send()` outputs strings/objects and sets content types automatically. `res.json()` explicitly converts inputs to JSON, while `res.end()` finishes the response process immediately without a body.

19. What is the purpose of CORS, and how do you resolve CORS errors in Express?

CORS is a browser security feature that blocks a frontend from fetching resources from a different backend domain. You resolve it by implementing the `cors` middleware to whitelist allowed frontend origins.

20. How do you serve static assets like images, CSS, or frontend build files in Express?

You use the built-in `express.static()` middleware helper to serve static files from a specific folder. For example, `app.use(express.static('public'))` exposes everything in the public directory to the root URL.

Section 3: REST APIs & HTTP Protocol

21. What is a REST API, and what makes an API truly "RESTful"?

A REST API is an architectural style for building network systems using stateless client-server communication. It is truly RESTful when it maps standard HTTP methods to clean, resource-oriented endpoint URLs.

22. Explain the precise difference between the PUT and PATCH HTTP methods.

`PUT` replaces an entire target resource with the provided update payload, wiping out unmentioned fields.

`PATCH` applies partial modifications, updating only the specific properties included in the payload.

23. What do the HTTP status codes 200, 201, 400, 401, 403, and 500 represent?

`200` means success, `201` means created, and `400` indicates a bad client request. `401` means unauthorized (needs login), `403` means forbidden (no permission), and `500` is an internal server error.

24. What does it mean when we say HTTP is a "stateless protocol"?

It means the server treats every incoming request as entirely new and independent. The backend retains no contextual memory or historical awareness about previous requests from that client out of the box.

25. What is the difference between a Monolithic architecture and Microservices?

A Monolith houses all application features, routing, and database logic inside a single codebase on one server. Microservices break an app into small, independent services communicating over the network.

Section 4: Security, Authentication & Session Management

26. What is a JWT (JSON Web Token), and how is it structured?

A JWT is a compact string used to share authenticated user claims securely between clients and servers. It is structured into three parts separated by dots: a Header, a Payload, and a Signature.

27. Where should a frontend client store a JWT, and what are the security trade-offs?

`localStorage` is vulnerable to Cross-Site Scripting (XSS) script theft but easy to implement. `httpOnly` secure cookies are much safer because they block browser JavaScript from reading the token directly.

28. What is the difference between Authentication and Authorization?

Authentication is the process of verifying *who* a user is (e.g., checking credentials during login). Authorization is verifying *what* that authenticated user is allowed to do (e.g., admin permissions).

29. What is a Cross-Site Scripting (XSS) attack, and how do you protect a Node backend?

An XSS attack happens when malicious scripts are injected into web pages via unvalidated user input fields. You protect the backend by sanitizing user inputs and setting security headers with the `helmet` package.

30. What is a CSRF (Cross-Site Request Forgery) attack, and how does it exploit cookies?

CSRF tricks a victim's browser into executing an unauthorized, state-changing command on a backend using saved session cookies. You prevent it by checking anti-CSRF tokens or using `SameSite=Strict` cookie attributes.

31. Why should you never store plain-text passwords in a database? What library helps avoid this?

Plain-text passwords expose accounts immediately if a database leak occurs. You use the `bcrypt` library to securely hash passwords into irreversible strings before saving them.

32. What is "Salting" a password, and why is it necessary?

Salting means appending a unique, random string of characters to a password before hashing it. It ensures identical user passwords yield completely unique hashes, blocking attackers from using pre-computed crack lists.

33. What is Rate Limiting, and why should you use it on your API routes?

Rate Limiting restricts the maximum number of requests an IP address can make within a specific timeframe. You use it to protect servers from Denial-of-Service (DoS) and brute-force hacking attempts.

Section 5: Practical Operations & Scenario-Based Questions

34. What is Environment Variable configuration, and why do we use dotenv?

Environment variables store sensitive keys outside source code to protect credentials. The `dotenv` package loads these configurations from a local `.env` file into Node's runtime `process.env` object.

35. Explain the difference between horizontal and vertical scaling for a backend server.

Vertical scaling involves adding more hardware power (RAM/CPU) to a single existing server instance. Horizontal scaling involves adding entirely new clone server machines to distribute the incoming traffic load.

36. What is the Node.js cluster module, and how does it optimize CPU utilization?

The `cluster` module allows you to spawn multiple duplicate worker processes that share the same network ports. It optimizes performance by spinning up one worker process per available CPU core.

37. What is a Memory Leak in a Node.js application, and how can it happen?

A memory leak occurs when an app retains references to unused objects, preventing memory cleanup. It happens due to uncleaned global variables, active intervals, or unresolved event listeners.

38. How do you handle unhandled promise rejections or uncaught exceptions globally to prevent crashes?

You listen to process-level lifecycle events by registering global listeners for `uncaughtException` and `unhandledRejection`.

```
process.on('uncaughtException', (err) => { log(err); process.exit(1); });
```

39. What is an Event Emitter in Node.js, and what is its basic usage pattern?

An Event Emitter is a class object that allows you to trigger named events and handle custom real-time signals. It uses `.on()` to register event listeners and `.emit()` to broadcast the trigger signal.

40. What is PM2, and why would you use it when deploying a Node application to production?

PM2 is a production-grade process manager for Node.js. You use it because it automatically restarts your app if it crashes, manages system log files, and supports zero-downtime reloads.

41. What are WebSockets, and how do they differ from traditional HTTP polling?

HTTP polling requires the client to repeatedly send requests to check for data. WebSockets establish a permanent, bidirectional communication channel over one TCP connection for instant, real-time data streaming.

42. Explain the difference between `npm install` and `npm ci`.

`npm install` updates packages and can modify the lockfile based on version rules. `npm ci` ignores `package.json` and strictly installs the exact versions in `package-lock.json`, ideal for CI/CD.

43. What is the purpose of the cluster master and worker model?

The master process is solely responsible for monitoring, spawning, and managing child nodes. The worker processes handle the actual network workload, processing API requests and application logic.

44. What is a Callback Hell, and how do Promises or Async/Await resolve it?

Callback Hell occurs when deeply nested asynchronous functions become unreadable and unmaintainable. Promises flatten this structure into linear chains, and Async/Await writes them like synchronous code.

45. What is Object-Relational Mapping (ORM) or an Object Document Mapper (ODM)?

It is a library that allows you to interact with databases using regular JavaScript objects and methods. It eliminates the need to write raw database queries (e.g., Sequelize for SQL or Mongoose for NoSQL).

46. What is structural Logging, and why is `console.log` discouraged in production?

`console.log` is synchronous, blocks execution, and prints unstructured text. Production loggers like Winston output structured JSON streams that are easy for analytics software to filter and parse.

47. How do you implement input validation in Express safely without messy logic structures?

You implement validation by placing third-party validator packages like `joi` or `express-validator` as middleware layers directly in front of your route controllers to reject malformed requests early.

48. What is the difference between asymmetric and symmetric encryption?

Symmetric encryption uses the exact same single secret key to both encrypt and decrypt data assets. Asymmetric encryption uses a public key to encrypt data and a private key to decrypt it.

49. What is a Connection Pool, and why is it used for database connectivity?

A connection pool maintains a cache of active database connections that can be reused for incoming traffic. This removes the heavy performance overhead of opening and closing connections on every API request.

50. Imagine backend response times increase from 100ms to 8 seconds during peak hours. What are your first debugging steps?

First, check server CPU, RAM, and disk metrics to spot hardware bottlenecks. Second, inspect database logs to identify unindexed queries or a bottlenecked database connection pool.